॥वसुधैव कुटुम्बकम्॥

SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

B-Tech Project

ForgeFlow: An AI-Native End-to-End Software Development Lifecycle Automation Platform

Group Number: Group – 04

Raghav Sonchhatra – PRN 23070122172

Sanidhya Awasthi – PRN 23070122192

Faheemuddin Sayyed – PRN 23070122196

Guided By:

Dr. Payal Kadam

Professor

Computer Science

CONTENTS

Roadmap for Review 4

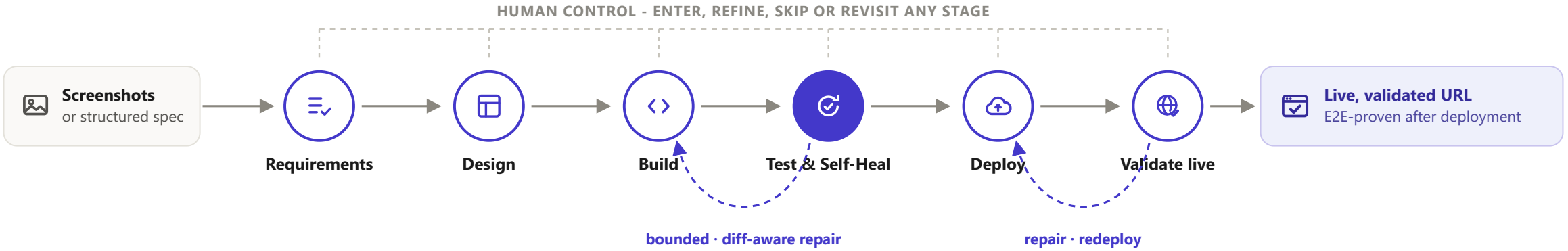
01	Introduction	03
02	Motivation	04–05
03	Status of Current Work	06
04	Literature Review & Research Gaps	07–12
05	Prototype / Software Development	13–15
06	Testing & Performance Validation Report	16–19
07	Gantt Chart / Project Tracking	20
08	Collaboration Evidence & Peer Reviews	21
09	Conclusion	22
10	References	23–24

> “ Review 1 showed the pipeline exists. Review 2 showed why it is designed this way. Review 3 shows it **verified end-to-end** - with the evidence - and delivery of the platform itself being automated.

INTRODUCTION

ForgeFlow - An AI-Native, Human-in-the-Loop Platform for the Automated Web SDLC

Screenshots or a spec in → a tested, deployed website out.



Verified end-to-end. The full hero path has been run on real deployments - frontend and backend on Vercel, data on MongoDB Atlas - and re-validated on the live URL.

No stage is mandatory - and now no skip is final. Skips are reversible through restore points, and a human can approve a Build instead of paying for a re-run.

Core contribution, sharpened: the bounded, diff-aware repair loop now follows the failing test's import graph, so it can reach the file that actually holds the bug.

Now: delivering ForgeFlow itself continuously. A 6-job CI already gates every push and pull request; continuous deployment of the platform is in progress.

> Existing tools generate apps. ForgeFlow generates apps it can *prove* work - and then proves it again on the live URL.

Adoption Keeps Rising - Trust Does Not Follow

Adoption keeps rising

84% of developers use or plan to use AI tools - **up from 76%** a year earlier (49,000+ respondents) [1]

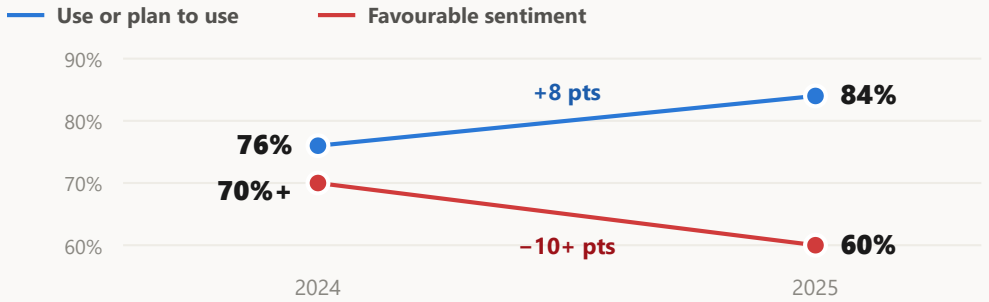
90% of ~5,000 technology professionals now use AI at work; over 80% believe it made them more productive [2]

51% of professional developers use AI tools **every day** [1]

31% already use **AI agents** at least monthly - the shift from autocomplete to delegated work has begun [1]

Trust is not following

Use of AI tools vs. favourable sentiment toward them [1]



2026 survey results not yet published; 2025 is the latest release.

46% vs 33% **distrust vs. trust** the accuracy of AI output - only 3% trust it highly [1]

66% name “almost right, but not quite” their top frustration; 45% say debugging AI code takes longer [1]

+19% task time for experienced developers in an RCT when AI was allowed - while they **believed they were 20% faster** [3]

MOTIVATION (2/2)

The Damage Has Moved Downstream - Into Delivery and Production

81%

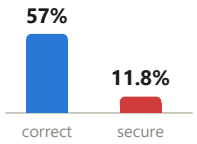
of enterprise technology leaders report **more production incidents caused by AI-generated code** [4]

70% say maintaining test suites is now a bigger burden than writing code [4]

20.8–30.2%

first-attempt deployment success when frontier LLMs generate cloud infrastructure - **deployability fails separately from code quality** [6]

FSE 2026 · 153 real-world scenarios, 58 cloud services [6]



57% of a leading agent's solutions to real feature requests worked - only 11.8% were secure (186 tasks, ICML 2026); vulnerability hints did not fix it [5]

18.9%

of **real CI failures** fixed by the best LLM when every fix is validated by re-running the full CI workflow (567 failures, 103 repositories) [7]

Stability ↓

AI adoption now lifts delivery throughput - but **still correlates negatively with delivery stability** [2]

Where ForgeFlow answers each: incidents after shipping → the same Playwright suite is re-run against the **live URL** · deployability → infra analyzer + a deploy that is truthful about env, logs and teardown · hard CI failure classes → environment failures never reach the repair agent · test-suite burden → tests are generated from the requirements and traced back by `criterion_id`.

> Generation is solved. **Verification is not** - and delivery is where the gap now shows.

STATUS OF CURRENT WORK

Pipeline Verified End-to-End - Now Hardening Delivery Itself

Upload screenshots / spec → Capture requirements → Refine design → Generate app → Generate & run tests → Self-heal failures → Deploy → Validate live URL

63 / 63	2,148	✓ VERIFIED	6 jobs · 21	◑ IN PROGRESS
PHASES SHIPPED · 13 EPICS	AUTOMATED TESTS PASSING · 19 SEP	END-TO-END HERO PATH · LIVE URL	CI GATES ON EVERY PUSH / PR	CONTINUOUS DEPLOYMENT

SUBSYSTEMS - SHIPPED, AND WHAT CHANGED SINCE REVIEW 2

NEW SINCE R2

Control plane
Conductor + stage machine; skips reversible, Build human-approvable

PWA + web IDE
Monaco, xterm.js, live preview; resizable, focusable workspace

NEW SINCE R2

Execution sandbox
Per-project Docker; dependency-restart detection & diagnostics

NEW SINCE R2

Design stage
Stitch / Figma; a provider's question now waits for the user

Build stage
Phased plan → verify → bounded self-heal; stale-demo-test stop

Requirements → tests
Structured capture → unit + Playwright; sandbox & live runs scoped

CORE · UPGRADED

Self-healing repair loop
Import-graph context, bounded file request, no-op guard

NEW SINCE R2

Deploy
Vercel FE + BE, Atlas; env mirrored, real logs, teardown

Live validation
Playwright suite re-run on the live URL - verified end-to-end

NEW SINCE R2

LLM transport & cost
Provider-agnostic; cheapest-upstream routing; ₹ budget caps

Data & evaluation
Per-project DBs + CRUD browser; headless harness, 10 specs

IN PROGRESS

Platform CI/CD
6-job CI live on every push; CD for ForgeFlow in progress

✓ **New since Review 2:** verified on real deployments - every defect it surfaced became a fix plus regression tests (**+248 tests**).

Generation Is Mature - Deployability and Human Control Are Not

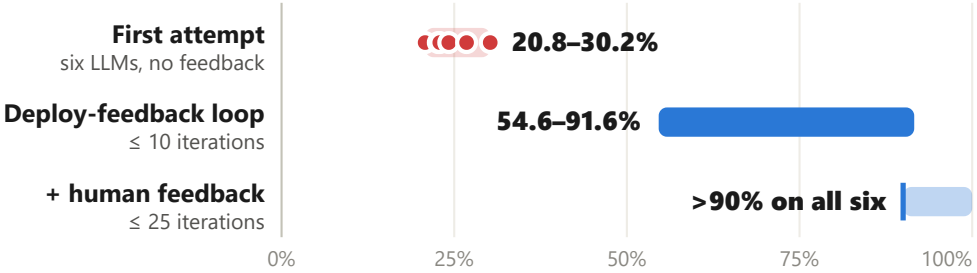
Agentic generation & issue resolution - a solved-enough problem

LLM4SE survey, SCIS'26 [8]	**926 studies** systemised across **112 SE tasks and 5 lifecycle phases** - LLM use across the lifecycle is no longer in question.
Issue-resolution survey [9]	**242 recent studies** organised into benchmarks, techniques and empirical work; reinforcement learning is the rising training paradigm.
Agentless, FSE'25 [10]	**Structured beats autonomous:** a fixed localise → repair → validate pipeline topped open-source agents on SWE-bench Lite (32.0%) at about $0.70 per issue.
RepairAgent, ICSE'25 [11]	An agent that plans its own tool calls fixed 164 Defects4J bugs (39 never fixed before) - at ~270k tokens per bug.
Huang et al., TSE'25 [12]	Larger code models repair better, yet strategies that helped earlier models **show no advantage on LLMs** - a repair recipe must be re-validated, not assumed.

Design consequence: ForgeFlow takes the **Agentless** lesson, not the autonomous-agent one - a fixed, deterministic stage pipeline, with a hand-written 53-file app skeleton *copied* rather than generated.

The deployed artifact still fails - until feedback is added

Deployable IaC, 6 LLMs · 153 scenarios · 58 cloud services [6]



… and the human stays in the loop

HULA, ICSE-SEIP'25 [13]	Deployed in Atlassian Jira: engineers refine and guide the agent's plan and code - less effort, though code quality remained a concern.
HULA follow-up [14]	Names the open challenges: the **cost of correctness testing** and the variability of LLM-based evaluation.
Design2Code, NAACL'25 [15]	484 real webpages: models still lag on visual-element recall and layout - so design stays **pluggable and human-refinable**.

> The field converged on generation. It did not converge on **deploying successfully, or on keeping the human in charge**.

Iteration Is Not Free - and Generated Tests Decay When Code Changes

Each extra round carries a risk

Security drift, ISTAS'25 [16]

400 samples, 40 rounds of "improve it": critical vulnerabilities **rose 37.6% after five iterations**; the authors call for human validation between rounds.

Build code, MSR'26 [17]

Agent-written build code carried **364 maintainability and security smells** - yet over 61% of those PRs merged with minimal human review.

Test generation - proven, on a program that holds still

Unit tests, EMSE'26 · ISSTA'25 [18][19]

Across 216,300 generated tests, hallucinated symbols caused **up to 86% compile failures** [18] - even though fine-tuned LLMs beat prior tools on all three unit-test tasks [19].

E2E & coverage, ICSE'25–'26 [20][21]

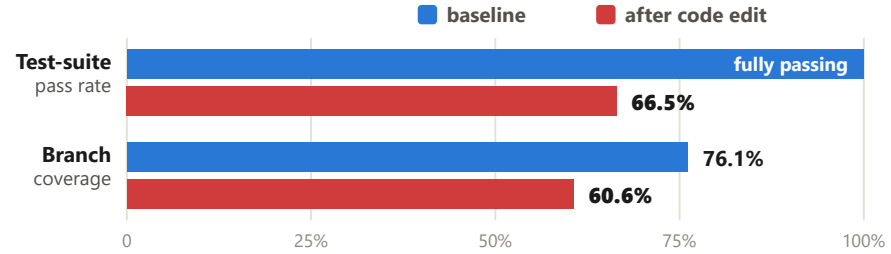
Feature-driven E2E tests reach 79% feature coverage on web apps; analysis-guided iteration gives 26% more line and 23% more branch coverage.

TraceLLM, RE'26 [22]

Recovers requirement-to-artifact links after the fact; ForgeFlow builds the link in **by construction** via `criterion_id`.

LLM tests under software evolution - arXiv, Mar 2026 [23]

8 LLMs | 22,374 program variants | ≈346M tokens



>99% of the failing tests **pass on the original program** - the model tests the code it *remembers*, not the code it was *given*.

> *"Computing a code diff and explicitly highlighting the modified regions" is the authors' recommended fix.*
>
> Haroon, Khan & Gulzar [23]

This is **ForgeFlow's diff-aware minimal repair context** - designed and built, then independently corroborated by this study.

> " Unbounded rounds erode code, and tests written for old code mislead - **bound the loop, and show the model the diff**.

No Existing System Verifies, Deploys, Validates Live - and Stops

	Research agents [9]–[11]	LLM test / repair tools [12], [18]–[21]	Commercial builders [24]	ForgeFlow
Multimodal design intake (screenshot → UI)	✗	✗	✓	✓ pluggable
Full-stack app generation	◑	✗	✓	✓ fixed stack
Requirements → executable tests (traceable)	✗	◑	✗	✓ structured
Bounded repair - iteration cap + stall detection	✗	✗	✗	✓
Diff-aware minimal repair context	✗	◑	✗	✓ + import graph
Human escalation on non-convergence	✗	✗	✗	✓
Automated cloud deployment	✗	✗	✓ locked-in	✓ dual-mode
Post-deployment live validation	✗	✗	✗	✓
Non-linear, skip-any-stage control	✗	✗	◑	✓ reversible
Per-run cost accounting + budget caps	✗	✗	✗	✓

✓ supported ◑ partial ✗ absent - highlighted rows: where the whole field says no

Research systems verify but do not deploy. Industry systems deploy but do not verify. **Neither validates the live artifact - and none bound the repair loop.** The 2026 evidence on each gap follows.

Agents Now Ship Into CI/CD - and CI Is Where Their Work Is Judged

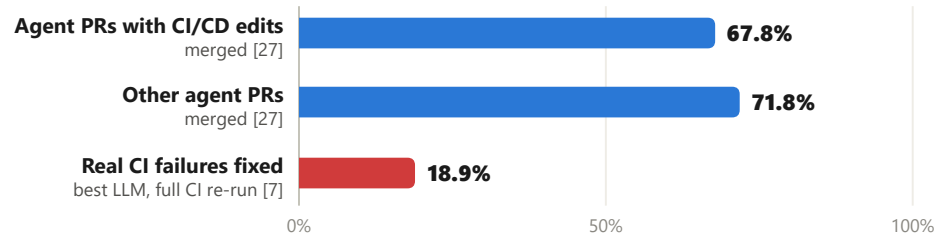
Agentic pull requests at scale - MSR 2026 evidence

AIDev dataset [28]	**456,000+ agent PRs** from 5 agents across 61,000 repositories: agents are faster than humans, yet their PRs are **accepted less often**.
Failed agentic PRs [25]	33k PRs: rejected ones are larger, touch more files and **often fail the project's CI/CD validation**; bug-fix tasks fare worst.
Merged or rejected? [26]	Only 35.7% of rejections are clear agent failures; **31.2% are workflow constraints** - process decides as much as code.
Agents & CI/CD config [27]	CI/CD edits are just 3.25% of agent changes (96.77% of them GitHub Actions), merged at 67.8% vs. 71.8% for other changes.

Design consequence: ForgeFlow holds its **own** delivery to the gate it applies to generated apps - nothing reaches `main` without the 6-job CI, and CD will only promote a green build.

Fixing what CI rejects is still hard

Agent PR outcomes vs. real CI repair [7][27]



CI-Repair-Bench [7]	567 real failures, 12 error types: formatting & lint get fixed; **environment, dependency and configuration failures stay hard**.
DORA 2025 [2]	AI amplifies what is already there: teams with **fast feedback loops and strong safety nets** gain; weak pipelines ship poor work faster.

Matches our classifier: environment / sandbox failures short-circuit with **zero** repair iterations; only code-class failures ever reach the repair agent.

> In 2026 an agent's work is judged by whether CI accepts it. ForgeFlow now applies that bar to **its own delivery**, not just to the apps it generates.

Repair Fails on Context, Not the Model - and Its Gains Stop Early

Context decides whether a repair is even possible

15–17× gain from **file-level** localization over a no-file baseline; best with ~6–10 relevant files, while line-level expansion often **adds noise** (500 SWE-bench Verified tasks, 61 configurations) [29]

RepoGraph, ICLR'25 [30]	A repository-level code graph - lines as nodes, dependencies as edges - retrieved as k-hop ego-graphs lifts SE agents.
LocAgent, ACL'25 [31]	Dependency graphs plus multi-hop search reach **92.7%** file-level localization accuracy.
Tests under evolution [23]	>99% of failing LLM tests still pass on the original program - the model tests code it remembers, so the diff must be highlighted.

Design consequence (phase 63): a deterministic **import-graph walk** from the failing test (depth ≤ 3, ≤ 24 files) replaces directory-sibling widening - graph-shaped context as in [30][31], with no extra agent turns.

… and the repair loop has an economics

> *"The first three to four repair iterations account for most achievable gains, while later iterations contribute only marginal improvements."*
>
> **Kiecker et al. [32] · Arimbur [33]: across seven models, most gains land in the first two rounds**



checked every iteration: **regressions ≥ 2 · stall 2 · no-op 2 · budget** - first hit ends the loop

Context quality [29]	Quality outweighs quantity - hence the per-file (8k) and whole-context (24k-char) caps, with every dropped part counted, never silent.

Design consequence: cap = 5, stall = 2 - plus a **new no-op guard**: two attempts that write nothing escalate as `no_patch` with the agent's own explanation, instead of re-asking the same question.

> The 2026 evidence says: give the model **the right file**, and **stop early**. Phase 63 does both.

Ambiguity Needs a Question, Not a Guess - and a Human in Charge

Models guess when the brief is ambiguous

ClarifyCodeBench [34] — Strong code generators are not strong clarifiers; clarification **degrades sharply** as ambiguity density rises.

Orchid, 1,304 tasks [35] — Ambiguity degrades every model - most of all the most advanced - and yields **functionally divergent** code for one request.

… and users must stay in charge

20,574 sessions [36] — **91.49%** of visible misalignment resolutions still needed explicit user correction.

Practice & roadmap [37][38] — Professionals control rather than delegate; the SASE roadmap calls for **agent-initiated human callbacks** on ambiguity.

Design consequence: a design provider's question is parked as a `DesignQuestion`, the stage goes `awaiting_user`, and the answer resumes the same provider - no silent failover to a design nobody asked for.

Gap → 2026 evidence → ForgeFlow response (shipped since Review 2)

GAP	EVIDENCE	FORGEFLOW RESPONSE
Agent work is judged by CI, and CI repair rarely succeeds	[7][25][27]	**6-job CI on every push**; platform CD in progress
The right file matters more than more context	[29]–[31]	**Import-graph repair context** + bounded `request_file` (≤ 3)
Repair gains stop after 3–4 iterations	[32][33]	Cap 5 · stall 2 · regressions 2 · **new no-op guard 2**
Models do not ask when a brief is ambiguous	[34][35]	**Provider questions routed to the user**
Users must correct and override agents	[36]–[38]	**Reversible skip**; human-approved Build
Deployment fails apart from code quality	[6]	Env mirrored to provider, real logs, DB pre-flight guard, teardown
Cost of verification is an open challenge	[14][39]	Role-based model routing + **cheapest-upstream routing**; ₹ budget caps

Research systems verify but do not deploy; builders deploy but do not verify. ForgeFlow does both and **stops on a bound** - and every 2026 gap above now maps to shipped code.

What Was Built Since Review 2 - Every Change Traced to a Defect

CHANGE · AREA	DEFECT OR GAP IT CLOSES → WHAT NOW HAPPENS	TESTS
Stale demo-test guard · Build	Skeleton demo tests asserted placeholder copy the build gate forbids, and tests are read-only to repair - an unwinnable loop. Now a **zero-iteration, zero-token stop** that names the files.	+8
Reversible skip + human build approval · Conductor	Undoing a skip meant a paid rebuild. **Restore points** bring the previous status back, and a human can approve a Build instead of re-running it.	+29
Phase 62 · Deploy truthfulness	Env vars shown as set were absent on Vercel; "Logs" showed ForgeFlow's own log; no teardown. Env now **mirrored to the provider**, real provider logs, BYO readiness strip, deployment delete, localhost-DB pre-flight guard.	+63
Phase 63 · Repair context reachability	Repair was handed `main.tsx`, not the component under test. **Import-graph walk**, per-test coverage, bounded `request_file`, `no_patch` escalation (PR #3).	+28
Deploy & test-runner hardening	Analyzer read failures, a dedicated test database, frontend-builder exit codes, and build-failure handling inside deploy orchestration.	+35
Sandbox dependency restart · Execution	After a Docker restart the app-DB host stopped resolving (`EAI_AGAIN`) and read like a config typo. Now detected, diagnosed, dependencies restarted.	+13
Price-sorted routing · LLM transport	On OpenRouter one model id is served by several upstreams at different prices → request **cheapest-first** with fallbacks kept; every other endpoint gets a byte-identical body.	+9
Coverage refactor · sandbox, testing	Shared sandbox fixtures, and an end-to-end test that pressing **Repair** really patches code.	+21
Test runs scoped by environment	A live-validation run (E2E subset only) could masquerade as the Test stage's latest result. Sandbox runs by default, explicit `env` filter.	+2
Design clarifying questions	A provider's question was treated as an error, so failover produced a design nobody asked for. Question parked; stage `awaiting_user`; the answer resumes the same provider.	+18
Platform continuous deployment	CI verifies every push, but nothing yet deploys ForgeFlow itself - in progress (slide 15).	-

248 new test definitions since the Review 2 cut (189 backend · 59 frontend), including datetime formatting (+16) and pane sizing (+6), not listed. Source: git history.

Core Contribution, Upgraded - Repair Reaches the File Under Test

BEFORE
directory-shaped
up to Review 2

Failing spec
frontend/src/App.test.tsx
read-only oracle

→

Same-folder sources
main.tsx · routes.tsx · …
sorted → main.tsx first

→

Model receives main.tsx
"I cannot fix this from main.tsx alone"

→

Writes nothing
identical context
re-sent, twice

→

Stall → escalate
iterations spent,
bug never reached

Live session, 30 Aug - *Simple Todo App*, 10 / 13 tests passing: the component under test lives one directory away, in `pages/`.

AFTER
import-shaped
phase 63 · 30 Aug

Failing spec
App.test.tsx
still read-only

→

Import-graph walk
import { HomePage }
from './pages/HomePage'

→

Rank + per-test coverage
direct imports > folder neighbours
re-checked after every tier

→

Minimal context
≤ 24k chars + git diff
since last green + criterion

→

Repair agent
reaches
pages/HomePage.tsx

depth ≤ 3 · ≤ 24 files · test files filtered out

a tier that adds an irrelevant file no longer blocks the tier that holds the answer

+ request_file ≤ 3 / attempt · tests refused
2 empty attempts → no_patch + agent's reason

Every run ends in a named terminal state

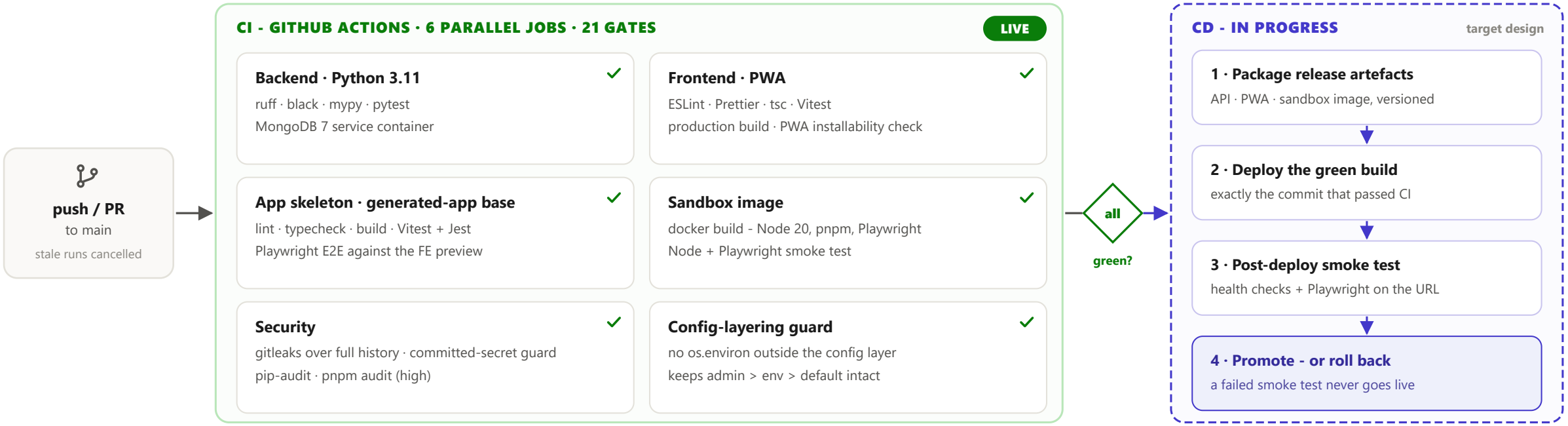
✓ fixed · regressed · stalled · no_patch · new · cap_reached · budget · blocked · cancelled

Escalation hands the user the failing tests, diffs tried, metrics and a resume contract - non-convergence is an outcome, never a silent failure.

NEW BOUND (ADMIN-TUNABLE)	DEFAULT	EFFECT - 0 SWITCHES IT OFF, NO DEPLOY NEEDED
`repair_import_max_depth`	**3**	how far the walk follows imports
`repair_import_max_files`	**24**	sources one walk may add
`repair_request_file_max`	**3**	files the agent may request per attempt
`repair_max_noop_attempts`	**2**	empty attempts before `no_patch`

> "Minimality is strengthened, not traded away: "what the test imports" is more precise than "what sits beside it" - a deterministic step, not an extra agent turn [10].

Delivering ForgeFlow Itself - CI Gates Live, CD In Progress



Why CD must be gated

81% of leaders report more AI-linked production incidents [4]; AI adoption still correlates with lower delivery stability [2].

Same principle as the product

Generated apps are checked by their own generated tests in the sandbox, then again on the live URL. ForgeFlow's own releases get the same treatment.

Already caught by the gate

mypy reports 4 type errors in 2 test files from the 12 Sep design-questions commit - CD must not promote until they are fixed.

> ForgeFlow holds a generated app to its tests - in the sandbox and again on the live URL. Review 3 puts **the platform itself** under the same rule.

Automated Verification - 2,148 Tests Passing, Every Static Gate Run

Test-suite results - full run, 19 Sep 2026, development host (Windows 11)

SUITE	COLLECTED	PASSED	FAILED	SKIPPED	WALL TIME
Backend · pytest	1,713	**1,642**	2 [1]	69 [2]	674–874 s
Frontend · Vitest, 61 files	506	**506**	0	0	331 s
Total	**2,219**	**2,148**	**2**	**69**	-

[1] Both need a running Docker daemon, which was off on the reporting host ("Sandbox runtime unavailable") - an environment condition, not a code defect; CI runners have Docker. [2] 66 POSIX-only tests (pty, process groups, `pgrep`) run on the Linux CI runners; 3 opt-in real-toolchain tests are covered by the CI skeleton job. A second full run reproduced the same result exactly.

Static quality gates - same host, same day

✓ ruff · ✓ black · 419 files · ✓ tsc · strict · ✓ ESLint · 0 warnings · ✓ Prettier

✕ mypy · 4 errors, 2 test files - open

The mypy errors (`Any | None` indexed in two new design tests, 12 Sep) are real and would fail the CI backend job; they are tracked as the first item that CD must not promote past.

Growth since the Review 2 cut (24 Aug → 12 Sep)

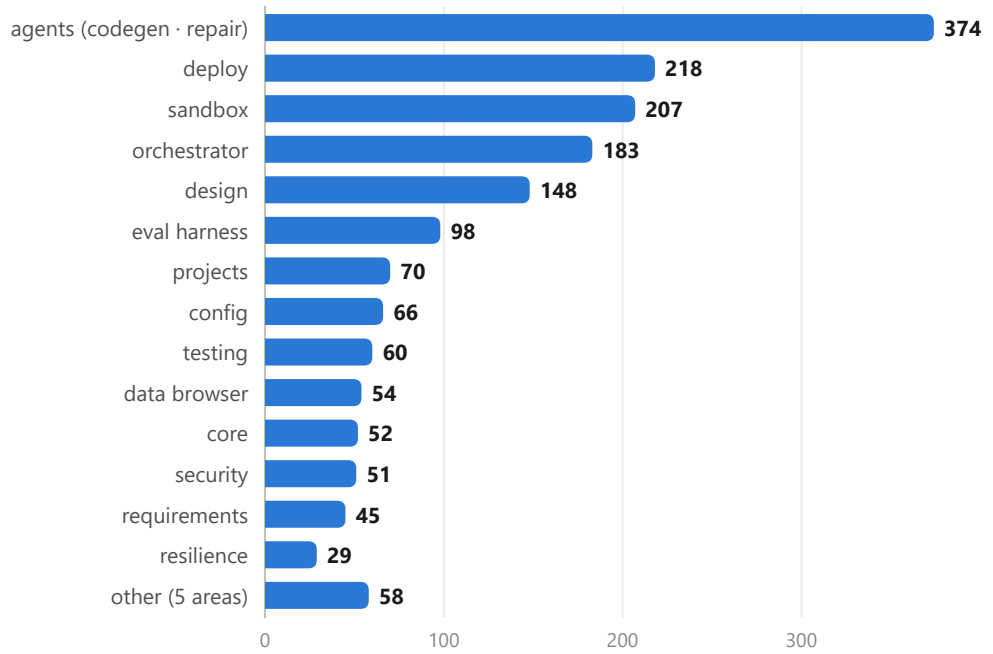
1,726 → 1,955
TEST DEFINITIONS (+13%)

+5,433
LINES OF TEST CODE

+248
TESTS ADDED WITH FIXES

Test definitions counted the same way at both commits (`def test_` / `it(` / `test(`); the runner's collected count is higher because parametrised cases are counted individually.

Backend tests by subsystem - 1,713 collected



Parametrised cases counted individually. "Agents" covers codegen, test generation, the repair agent and its context builders; "other" = realtime, demo, db, auth, root.

End-to-End Validation on Real Deployments - and What It Found

✓ Requirements	✓ Design	✓ Build	✓ Test	✓ Repair	✓ Deploy	✓ Validate
free text → spec → acceptance criteria	Stitch screens; questions go to the user	install → typecheck → boot → placeholder check	Vitest · Jest · Playwright in the sandbox	bounded, diff-aware, import-graph context	Vercel FE + BE · MongoDB Atlas	Playwright re-run on the live URL

#	OBSERVED IN A LIVE SESSION	ROOT CAUSE	FIX NOW IN PLACE
1	Env vars shown as *set* in ForgeFlow, absent from the Vercel dashboard	`set_env` built and unit-tested, never wired	Env mirrored to the provider project; readiness strip shows presence + last 4 only
2	"Logs" tab showed ForgeFlow's step log, not Vercel's	Provider deployment ids never recorded	Ids recorded; real provider build & runtime logs
3	No way to delete a deployment	`destroy` never wired	Teardown returns Deploy to `empty`, restoring the validate⇐deploy prerequisite
4	Localhost DB URI shipped to a billed function that timed out every query	No pre-flight reachability check	DB step refuses a localhost cluster URI before anything is deployed
5	Repair wrote nothing, twice, then stalled	Directory-shaped context; coverage judged once, globally	Import-graph walk, per-test coverage, `request_file`, `no_patch`
6	Build loop spent its whole budget and still failed	Stale demo tests assert copy the gate forbids; tests are read-only	Zero-iteration stop naming the stale test files
7	Preview failed with `EAI_AGAIN` after a Docker restart	DB and proxy don't come back with the sandbox	Unresolvable DB host detected, diagnosed and restarted
8	Latest live run shown as the Test stage's result	Shared collection, unscoped listing	Sandbox runs by default; explicit `env` filter
9	A design nobody asked for appeared after the provider asked a question	Question classified as an error → failover	Question parked; stage `awaiting_user`; answer resumes the provider

Performance - Bounded by Construction, Measured Where Possible

Worst-case bounds per run - guaranteed by construction (defaults, admin-tunable)

SCOPE	BOUND	GROUNDED IN
Repair loop	≤ 5 iterations; stops at 2 stalls, 2 regressions or 2 empty attempts	gains concentrate in iterations 1–4 [32][33]
Repair context	≤ 24,000 chars (≤ 8,000 per file, ≤ 8,000 of diff); every drop counted	quality over quantity [29]
Import walk	depth ≤ 3, ≤ 24 files; ≤ 3 requested files per attempt	the right file, bounded reach [30][31]
Build	≤ 8 plan phases · ≤ 60 tool turns / phase · ≤ 2 typecheck fixes / phase · typecheck ≤ 600 s · whole build ≤ 3,600 s	no unbounded loop anywhere
Environment failures	0 repair iterations and 0 tokens - classified before the loop	env / config failures are the hard class [7]
Cost	Per-project and global ₹ caps, warning at 80%; cheap model for routing, codegen model only for code; cheapest upstream first	cost of verification [14][39]

Measured this cycle

674 s full backend suite, **1,713 tests** - best of two identical runs (874 s on the other), about 0.4–0.5 s per test

331 s full frontend suite, **506 tests** across 61 files (jsdom)

34.5 s slowest single backend test - the OpenRouter SDK call-signature check

What the harness records per run

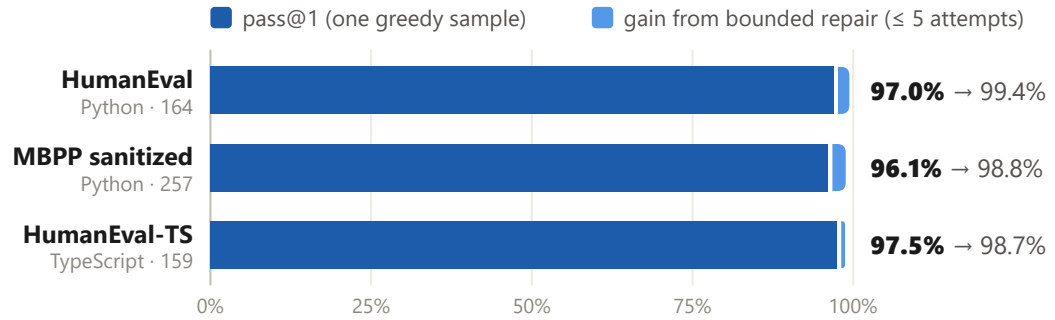
first-pass rate · pass after repair · repair delta · iterations · regressions introduced · tokens · ₹ · stage & total wall-clock · screenshot → URL time

Status: first scored run done – four industry benchmarks in full and 2 of 10 apps (next slide). The **three-arm study** - no repair · bounded diff-aware · context-ablated - is the next milestone.

> “ Today we report bounds and suite measurements. Convergence, cost per success and latency **with variance** come from the scored benchmark - a result we will measure, not assert.

Benchmarks - 97% HumanEval, 93% BFCL; Repair Adds up to 2.7 pp

Industry benchmarks · `z-ai/glm-5.3-flash` via ForgeFlow's LLM transport



pass@1 = one greedy sample, graded by the benchmark's own tests (leaderboard-comparable). Repair = failing output fed back, ≤ 5 attempts, stop after 2 without progress – ForgeFlow's loop policy.

Tool calling · BFCL v3 non-live (Python), official AST checker · mean **92.7%**



End-to-end apps · real pipeline · 2 of 10 specs (stopped for budget)

12/16	78/78	2/13
CRITERIA MET (TESTS COVER ALL)	UNIT TESTS PASS	E2E (PLAYWRIGHT) PASS

SPEC	CRITERIA MET	REPAIR TRIES	₹	MIN	OUTCOME
notes-crud	5/7 → **5/7**	0	21	34	escalated
bookmark-manager	7/9 → **7/9**	0	25	44	escalated

Repair made 0 attempts: every failure was an E2E test, which imports no app source, so the loop has nothing safe to patch and escalates by design. An isolated re-run of a third app's E2E suite passed 2/2.

Found by running it – fixed, each with a regression test

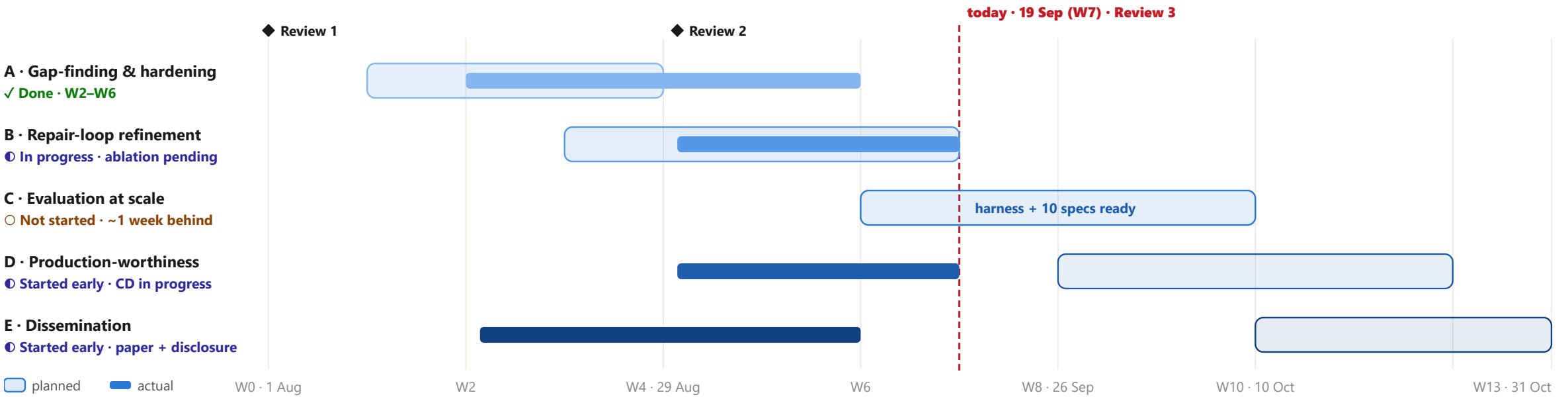
- **Test generation was never wired in** – pass rates had only measured skeleton placeholder tests.
- **Cheapest-first routing chose a 4-bit host** at ~3 tok/s – added an upstream pin.
- **E2E browsers crashed in the sandbox** – 3 workers hit the 256-pid cap; now 1 (peak 142).
- **Mid-stream timeouts escaped the retry loop** and failed whole builds.
- **Harness raced detached builds** – it now joins the build before measuring.

Why the numbers hold: grader self-check – 426/426 reference solutions pass, 580/580 wrong ones rejected; datasets pinned by commit + SHA-256; one model upstream for every call; code runs with no network. Reproduce: **make eval**.

Next: all 10 apps with fewer parallel builds, in the no-repair, bounded and context-ablated arms with repeated runs – and E2E-aware repair context.

GANTT CHART / PROJECT TRACKING

Project Tracking - Plan vs. Actual at Week 7 of 13



✓ Done since Review 2

- **Phases 62–63**: deploy truthfulness, repair reachability
- Reversible skip, human build approval, design questions
- Sandbox restart diagnostics; price-sorted routing
- Patent disclosure revised; paper figures revised

◑ In progress

- **Platform CD**: package → deploy green build → smoke → promote / roll back
- Clear the 4 mypy errors so the gate is green
- Repair-loop tuning ahead of the ablation

Next · W8–W13 (to 31 Oct)

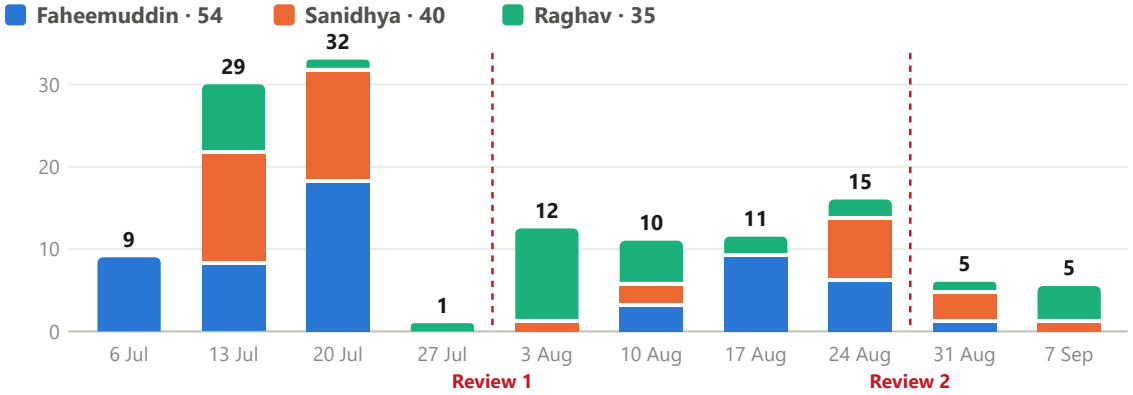
- **Scored three-arm benchmark** on the 10 specs, repeated runs for variance
- Expand to 20–30 specs; sweep iteration cap & stall threshold
- Results into the paper; disclosure submitted for filing

> A is done, B is in progress, C has slipped about a week, and D and E started early. **Evaluation is now the critical path.**

COLLABORATION EVIDENCE & PEER REVIEWS

Three Contributors, One Reviewed Workflow, Six IP-Review Fixes

Commits per week, by member (excl. merges)



Weekly commits per member, 11 Jul – 12 Sep 2026.

Faheemuddin Sayyed 54 commits
Deploy truthfulness (phase 62), repair reachability (phase 63), reversible skip & build approval

Sanidhya Awasthi 40 commits
Design clarifying questions, deploy & test-runner hardening, stale-demo-test guard

Raghav Sonchhatra 35 commits
Build verification & self-heal, patent-disclosure revision, paper figures

Branch → review → merge. Every change was developed on its own branch, reviewed by another teammate, then merged into `main`.

Spec and CI on every change. 63 phase specs with acceptance criteria; 6 CI jobs and 21 gates on every push and pull request.

IP review of the invention disclosure - six points, all applied

#	REVIEWER'S POINT	CHANGE APPLIED
1	Paying API / cloud costs does not establish ownership	Ownership governed by the **institutional IP Policy**; no external grant or sponsorship
2	Title too long	**"System and Method for Bounded Diff-Aware Self-Healing Software Application Delivery"**
3	Section 3(k) computer-program exclusion	**Technical effects foregrounded:** sandbox isolation, bounded context construction, diff-based repair, regression detection, deployment validation, credential isolation
4	Prior-art statement too categorical	"…the inventors have **not identified** a reference that discloses the claimed combination" (search to date)
5	Marketing-style absolutes	"Provable termination" → **"deterministically bounded termination under configured guard conditions"**

Outcome: claims narrowed to what the system demonstrably does - the same discipline applied to the paper, whose six figures were simplified and made vendor-neutral.

CONCLUSION

Verified End-to-End - Next: Measure It, Ship It Continuously

The adoption–trust gap persists: **84% adoption against 33% trust** [1]; 81% of leaders see more AI-caused production incidents [4]; the best LLM fixes only 18.9% of real CI failures [7].

The **full hero path has been verified end-to-end** on real deployments - 63 / 63 phases, 2,148 automated tests passing, 6 CI jobs gating every change.

Live validation **sharpened the core contribution**: repair now follows the failing test's import graph, with a bounded file request and a no-op guard - as 2026 localization evidence recommends [29]–[31].

Review became evidence: **six IP-review points** applied to the patent disclosure, and every live-session defect turned into a phase, a fix and regression tests (**+248**).

Now: continuous deployment of the platform, gated on a green CI - starting with the 4 open mypy errors the gate already caught.

Next: the three-arm benchmark on all 10 apps, with variance - the first scored run (97% HumanEval pass@1, 92.7% BFCL) already shows where repair helps and where it cannot reach.

> **Anyone can make an AI write an app. ForgeFlow knows when to stop - Review 3 shows it working end-to-end; next, we measure how often it succeeds.**

References

[1] Stack Overflow, "2025 Developer Survey - AI," Stack Overflow, Jul. 2025. [Online]. Available: https://survey.stackoverflow.co/2025/ai

[2] DORA, "State of AI-assisted Software Development 2025," Google Cloud, Sep. 2025. [Online]. Available: https://dora.dev/dora-report-2025/

[3] J. Becker, N. Rush, E. Barnes, and D. Rein, "Measuring the impact of early-2025 AI on experienced open-source developer productivity," arXiv:2507.09089, Jul. 2025.

[4] CloudBees, "The State of Code Abundance 2026," CloudBees, Inc., May 2026. [Online]. Available: https://www.cloudbees.com/lp/2026-state-of-code-abundance-report

[5] S. Zhao, D. Wang, K. Zhang, J. Luo, Z. Li, and L. Li, "Is vibe coding safe? Benchmarking vulnerability of agent-generated code in real-world tasks," in *Proc. 43rd Int. Conf. Mach. Learn. (ICML)*, 2026, arXiv:2512.03262.

[6] T. Zhang, S. Pan, Z. Zhang, Z. Xing, and X. Sun, "Deployability-centric infrastructure-as-code generation: Fail, learn, refine, and succeed through LLM-empowered DevOps simulation," in *Proc. ACM Int. Conf. Found. Softw. Eng. (FSE)*, 2026, arXiv:2506.05623.

[7] R. K. Muna, M. N. Rafi, and T.-H. Chen, "CI-Repair-Bench: A repository-aware benchmark for automated patch validation via CI workflows," arXiv:2604.27148, Apr. 2026.

[8] Q. Zhang *et al.*, "A survey on large language models for software engineering," *Sci. China Inf. Sci.*, vol. 69, no. 4, Art. no. 141102, Mar. 2026, doi: 10.1007/s11432-025-4670-0.

[9] Z. Jiang, D. Lo, and Z. Liu, "Agentic software issue resolution with large language models: A survey," arXiv:2512.22256v2, Aug. 2026.

[10] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Demystifying LLM-based software engineering agents," *Proc. ACM Softw. Eng.*, vol. 2, no. FSE, pp. 801–824, 2025, doi: 10.1145/3715754.

[11] I. Bouzenia, P. T. Devanbu, and M. Pradel, "RepairAgent: An autonomous, LLM-based agent for program repair," in *Proc. IEEE/ACM 47th Int. Conf. Softw. Eng. (ICSE)*, Ottawa, ON, Canada, 2025, pp. 2188–2200, doi: 10.1109/ICSE55347.2025.00157.

[12] K. Huang, J. Zhang, X. Bao, X. Wang, and Y. Liu, "Comprehensive fine-tuning large language models of code for automated program repair," *IEEE Trans. Softw. Eng.*, vol. 51, no. 4, pp. 904–928, Apr. 2025, doi: 10.1109/TSE.2025.3532759.

[13] W. Takerngsaksiri *et al.*, "Human-in-the-loop software development agents," in *Proc. IEEE/ACM 47th Int. Conf. Softw. Eng.: Softw. Eng. in Practice (ICSE-SEIP)*, Ottawa, ON, Canada, 2025, pp. 342–352, doi: 10.1109/ICSE-SEIP66354.2025.00036.

[14] J. Pasuksmit *et al.*, "Human-in-the-loop software development agents: Challenges and future directions," in *Proc. IEEE/ACM 22nd Int. Conf. Mining Softw. Repositories (MSR)*, Ottawa, ON, Canada, 2025, pp. 756–757, doi: 10.1109/MSR66628.2025.00112.

[15] C. Si, Y. Zhang, R. Li, Z. Yang, R. Liu, and D. Yang, "Design2Code: Benchmarking multimodal code generation for automated front-end engineering," in *Proc. 2025 Conf. Nations Americas Chapter Assoc. Comput. Linguist.: Human Lang. Technol. (NAACL)*, Albuquerque, NM, USA, 2025, pp. 3956–3974, doi: 10.18653/v1/2025.naacl-long.199.

[16] S. Shukla, H. Joshi, and R. Syed, "Security degradation in iterative AI code generation: A systematic analysis of the paradox," in *Proc. 2025 IEEE Int. Symp. Technol. Soc. (ISTAS)*, Santa Clara, CA, USA, 2025, pp. 1–8, doi: 10.1109/ISTAS65609.2025.11269659.

[17] A. Ghammam and M. Almukhtar, "AI builds, we analyze: An empirical study of AI-generated build code quality," in *Proc. 23rd Int. Conf. Mining Softw. Repositories (MSR)*, Rio de Janeiro, Brazil, 2026, pp. 727–731, doi: 10.1145/3793302.3793563.

[18] W. C. Ouédraogo *et al.*, "Prompt engineering in LLMs for automated unit test generation: A large-scale study," *Empirical Softw. Eng.*, vol. 31, no. 4, Art. no. 103, 2026, doi: 10.1007/s10664-026-10840-4.

[19] Y. Shang, Q. Zhang, C. Fang, S. Gu, J. Zhou, and Z. Chen, "A large-scale empirical study on fine-tuning large language models for unit testing," *Proc. ACM Softw. Eng.*, vol. 2, no. ISSTA, Art. no. ISSTA074, pp. 1678–1700, 2025, doi: 10.1145/3728951.

[20] P. Alian, N. Nashid, M. Shahbandeh, T. Shabani, and A. Mesbah, "Feature-driven end-to-end test generation," in *Proc. IEEE/ACM 47th Int. Conf. Softw. Eng. (ICSE)*, Ottawa, ON, Canada, 2025, pp. 450–462, doi: 10.1109/ICSE55347.2025.00141.

References

[21] S. Gu, N. Nashid, and A. Mesbah, "LLM test generation via iterative hybrid program analysis," in *Proc. IEEE/ACM 48th Int. Conf. Softw. Eng. (ICSE)*, Rio de Janeiro, Brazil, 2026, pp. 655–666, doi: 10.1145/3744916.3764553.

[22] N. Alturayeif, I. Ahmad, and J. Hassine, "TraceLLM: Leveraging large language models with prompt engineering for enhanced requirements traceability," *Requirements Eng.*, vol. 31, no. 1, Art. no. 6, 2026, doi: 10.1007/s00766-026-00460-1.

[23] S. Haroon, M. T. Khan, and M. A. Gulzar, "Evaluating LLM-based test generation under software evolution," arXiv:2603.23443, Mar. 2026.

[24] Y. Ge *et al.*, "A survey of vibe coding with large language models," arXiv:2510.12399, Oct. 2025.

[25] R. Ehsani, S. Pathak, S. Rawal, A. A. Mujahid, M. M. Imran, and P. Chatterjee, "Where do AI coding agents fail? An empirical study of failed agentic pull requests in GitHub," in *Proc. 23rd Int. Conf. Mining Softw. Repositories (MSR)*, 2026, arXiv:2601.15195.

[26] S. R. O. Peralta *et al.*, "Why are agentic pull requests merged or rejected? An empirical study," in *Proc. 23rd Int. Conf. Mining Softw. Repositories (MSR)*, 2026, arXiv:2605.22534.

[27] T. A. Ghaleb, "When AI agents touch CI/CD configurations: Frequency and success," in *Proc. 23rd Int. Conf. Mining Softw. Repositories (MSR)*, 2026, arXiv:2601.17413.

[28] H. Li, H. Zhang, and A. E. Hassan, "The rise of AI teammates in software engineering (SE) 3.0: How autonomous coding agents are reshaping software engineering," arXiv:2507.15003, Jul. 2025.

[29] M. Sepidband, H. V. Pham, and H. Hemmati, "On the role of fault localization context for LLM-based program repair," arXiv:2604.05481, Apr. 2026.

[30] S. Ouyang *et al.*, "RepoGraph: Enhancing AI software engineering with repository-level code graph," in *Proc. 13th Int. Conf. Learn. Represent. (ICLR)*, Singapore, 2025.

[31] Z. Chen *et al.*, "LocAgent: Graph-guided LLM agents for code localization," in *Proc. 63rd Annu. Meeting Assoc. Comput. Linguist. (ACL)*, Vienna, Austria, 2025, pp. 8697–8727.

[32] T. Kiecker, E. Reichmann, H. Kang, G. An, and L. Grunske, "Is three the magic number? An empirical evaluation of LLM-based repair loops," arXiv:2607.05197, Jul. 2026.

[33] J. J. Arimbur, "How many tries does it take? Iterative self-repair in LLM code generation across model scales and benchmarks," arXiv:2604.10508, Apr. 2026.

[34] Z. Fang, D. Jin, Y. Dong, Y. Li, K. Zhang, Z. Jin, and G. Li, "ClarifyCodeBench: Evaluating LLMs on clarifying ambiguous requirements for code generation," arXiv:2607.00711, Jul. 2026.

[35] D. Yang *et al.*, "Clarity is not assumed: Understanding LLM-based code generation under ambiguous requirements," arXiv:2604.21505, Apr. 2026.

[36] N. Tang *et al.*, "How coding agents fail their users: A large-scale analysis of developer-agent misalignment in 20,574 real-world sessions," arXiv:2605.29442, May 2026.

[37] R. Huang, A. Reyna, S. Lerner, H. Xia, and B. Hempel, "Professional software developers don't vibe, they control: AI agent use for coding in 2025," arXiv:2512.14012, Dec. 2025.

[38] A. E. Hassan, H. Li, D. Lin, B. Adams, T.-H. Chen, Y. Kashiwa, and D. Qiu, "Agentic software engineering: Foundational pillars and a research roadmap," arXiv:2509.06216, Sep. 2025.

[39] I. Ong *et al.*, "RouteLLM: Learning to route LLMs from preference data," in *Proc. 13th Int. Conf. Learn. Represent. (ICLR)*, Singapore, 2025.